WEBRUM-07: Session Replay

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 7 of 10 | **Created:** March 2026 | **Last Updated:** 10/31/2026

Overview

Session Replay allows you to visually replay a user's interaction with your web application — seeing exactly what the user saw, where they clicked, how they scrolled, and when errors occurred. Unlike traditional analytics that show numbers, session replay provides qualitative context for quantitative data.

Table of Contents

1. How Session Replay Works
2. Data Privacy and Masking
3. Session Replay Configuration
4. Finding Replay-Eligible Sessions
    - When a Session Has No Replay
5. Correlating Replay with Performance
6. Identifying UX Issues via Replay
7. Third-Party Replay Integration
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Session Replay license
Session Replay Enabled	Configured for at least one web application
Permissions	`storage:user.events:read`, `storage:user.sessions:read`, Session Replay viewer permission
Previous Notebooks	WEBRUM-01: RUM Fundamentals, WEBRUM-04: Session Analysis

1. How Session Replay Works

Dynatrace Session Replay captures the visual state of the web page as the user interacts with it. It does **not** record video — instead, it records DOM mutations:

Recording Mechanism

1. **Initial Snapshot** — When a page loads, the RUM agent captures the full DOM structure and inline styles
2. **Mutation Recording** — As the user interacts, DOM mutations (element changes, attribute modifications, text updates) are recorded incrementally
3. **User Events** — Mouse movements, clicks, scrolls, and keyboard inputs are captured with timestamps
4. **Beacon Delivery** — Recorded data is sent to Dynatrace in compressed beacons
5. **Server-Side Reconstruction** — Dynatrace reconstructs the visual experience from the DOM mutations and events

Recording Modes

Mode	Description	Data Volume	Privacy Impact
Visual replay	Full DOM + CSS reconstruction	High	Highest — captures visible content
Resource capture	Stylesheets are captured and stored; images and fonts are retrieved from the original location at playback	Very high	Captures stylesheets
Minimal mode	DOM structure only, no external resources	Low	Lower — placeholder images

> **Important:** Session Replay data is stored separately from RUM metrics. In the classic model, the default retention for captured resources is 35 days. Replay storage counts against your session replay quota.
>
> **Update (SaaS 1.343, July 2026 — staged tenant rollout) — Session Replay reaches general availability on Grail.** Replay data is stored natively in the Grail data lakehouse (`fetch user.replays`), with the video player, timeline, and multi-platform support built on that storage. On Grail-based tenants, replay retention follows the Grail bucket configuration rather than the classic quota model above. The classic model remains current for tenants that have not yet received 1.343 or still store replay in the classic surface — verify which model your tenant uses before planning retention.
>
> **Sources:** Configure Session Replay Classic for web applications (DT docs) — *"Images and fonts aren't captured."*; *"The default retention time for captured resources is 35 days."*

2. Data Privacy and Masking

Session Replay captures real user interactions, which may include sensitive data. Proper masking is essential for compliance with GDPR, CCPA, HIPAA, and other privacy regulations.

Masking Options

Option	What is masked	When to use
Mask all	All texts, user input, attribute values, and images	Testing Session Replay, or confirming that confidential data is not collected
Mask user input	All user input, including options in list boxes. Default since Dynatrace 1.262	Confidential information comes only from user input
Allow list	Everything in *Mask all*, except the elements you specify	The documented recommendation for most applications
Block list	Only the elements you specify	You want to clear rules from the *Mask all* set, or add your own

Mask all, *Mask user input* and *Allow list* do not hide user interactions with elements. With *Block list*, you can choose to hide interactions with masked elements.

HTML Attributes for Granular Control

Apply masking directly in your HTML:

```
&lt;!-- Mask this node and its descendants. RUM JavaScript 1.345+ also accepts
data-dt-mask --&gt;
&lt;div data-dtrum-mask&gt;
  &lt;p&gt;Sensitive content here&lt;/p&gt;
&lt;/div&gt;

&lt;!-- Unmask this element. Needs the allow-list entry described below --&gt;
&lt;span data-dtrum-allow&gt;Public content&lt;/span&gt;
```

`data-dtrum-allow` takes effect through the *Allow list* option: add one recording allow-list entry whose target element is the CSS selector `[data-dtrum-allow]`.

CSS Selector Masking Rules

From the application settings, go to **General settings > Data privacy > Session Replay > Content masking preferences**, select *Allow list* or *Block list*, and add the rules. Elements are defined by CSS selector. With *Block list*, selectors like these name what gets masked:

Selector	Effect
`.credit-card-form input`	Mask all inputs in credit card forms
`[data-sensitive]`	Mask elements with custom attribute
`.pii-container`	Mask PII containers

> **Warning:** Always err on the side of over-masking. It is better to mask non-sensitive content than to accidentally capture PII. Conduct a privacy review before enabling Session Replay in production.
>
> **Sources:**
>
> - Configure Session Replay Classic for web applications (DT docs) — *"Mask user input is the default masking option starting with Dynatrace version 1.262"*; *"you can use data-dt-mask instead of data-dtrum-mask"*; *"General settings > Data privacy > Session Replay"*
> - Session Replay for strong privacy requirements (DT docs) — *"Each HTML element that contains only allowed content can be marked with the attribute data-dtrum-allow"*

3. Session Replay Configuration

Sampling Rate

Not every session needs to be recorded. Configure sampling to balance insight with cost:

Scenario	Recommended Sample Rate
High-traffic production site	1-5%
Internal application	10-25%
Pre-production testing	50-100%
Investigating specific issue	100% (temporarily)

Conditional Capture

Use capture rules to record specific session types:

- **Error-triggered replay** — Automatically capture replay when JavaScript errors occur
- **Conversion-path replay** — Capture sessions that reach checkout pages
- **Frustration-triggered replay** — Capture sessions with rage clicks or exit intent

Storage Considerations

Factor	Impact on Storage
Page complexity (DOM size)	More complex pages = larger snapshots
Session duration	Longer sessions = more mutation data
SPA vs. traditional	SPAs generate more mutations from in-page updates
Resource capture enabled	Significantly increases storage (stylesheets)

4. Finding Replay-Eligible Sessions

Use DQL to identify sessions that have replay data available, then navigate to them in the Dynatrace UI.

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; frontend.name (stable; an array on user.sessions, so
expand it before grouping)
//                  primary_tags.application exists only if your OpenPipeline
adds it (corrected 10/14/2026)
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.value (page summaries;
web_vitals.time_to_first_byte where populated) —
//                  NOT ttfb.waiting_duration, which is the pre-request
wait/redirect phase
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the documented values are "real_user" / "robot" / "synthetic"
(lowercase);
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Sessions with replay data available in the last 24 hours
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(characteristics.has_replay) and characteristics.has_replay
== true
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| summarize {replay_sessions = count(),
    avg_actions = avg(user_action_count),
    avg_errors = avg(error.count)},
    by:{frontend.name}
| sort replay_sessions desc
```

```
// Find replay sessions with errors — prioritize these for review
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(characteristics.has_replay) and characteristics.has_replay
== true
| filter error.count &gt; 0
| fieldsKeep dt.rum.session.id, frontend.name, duration, user_action_count,
error.count, geo.country.name, browser.name
| sort error.count desc
| limit 20
```

```
// Replay coverage — what percentage of sessions have replay?
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| summarize {total_sessions = count(),
    replay_sessions = countIf(isNotNull(characteristics.has_replay) and
characteristics.has_replay == true)},
    by:{frontend.name}
| fieldsAdd replay_coverage_pct = round(toDouble(replay_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1)
| sort total_sessions desc
```

When a Session Has No Replay

The queries above filter for `characteristics.has_replay == true`. A `false` — or a null — is neither an error nor necessarily a misconfiguration. Replay is *conditional* by design, and several independent gates can each suppress it.

Cause	What actually happened	Where to check
Outside the sampling rate	In community practice, the most frequent cause. The effective recording rate is the RUM cost and traffic control **multiplied by** the Session Replay one — at 50% × 20%, 90% of sessions correctly have no replay	Both cost and traffic control settings for the application (§3)
Replay disabled for the application	Session Replay is configured per application, not globally — a tenant with replay enabled can still have applications without it	Settings → Session Replay, per application

Cause	What actually happened	Where to check
Unsupported browser	Session Replay publishes its own list of browser versions supported for recording, separate from RUM's; sessions from browsers outside it are not recorded	`browser.name` / `browser.version` on the session
A page blocked capture	URL exclusion rules stop recording on matching pages, sometimes on exactly the pages you care about. Masking rules hide matched content in the replay (§2)	Session Replay URL exclusion and data-privacy configuration
Quota exhausted	In community practice, where a consumption limit caps replay volume, eligible sessions stop being recorded once it is reached — confirm how your licensing applies before assuming this	Session Replay consumption

Forthcoming — SaaS 1.344 (released 07/27/2026, staged tenant rollout from 07/29/2026): Session Replay reports the reason itself. Users & Sessions gains specific information about why session recordings become unavailable, which collapses the elimination exercise below into a single read in the UI. Verify 1.344 has reached your tenant before relying on it; until then — and for any tenant still on an earlier sprint — the manual order below is the working path.

Eliminate in this order. In community practice the sequence that pays off is cheapest and most likely first, so you rarely reach the bottom.

1. **Sampling rate.** Compute actual replay coverage (the third query in this section) and compare it against the effective rate — RUM cost and traffic control × Session Replay cost and traffic control. If coverage ≈ that effective rate, **nothing is broken** — you are looking at sampling, and the fix is to raise the rate, not to debug.
2. **Application enablement.** If coverage is ~0% for one application while others are healthy, replay is off for that application. The coverage query already groups by application, so this shows up without extra work.
3. **Browser.** Compare `browser.name` and `browser.version` between sessions that do and do not have replay. A cause concentrated in one browser family is a support gap, not a configuration error — and not something you can configure away.
4. **Quota.** Check replay consumption last, and only after confirming a limit applies to you. A quota that ran out mid-period has a distinctive signature: replay present in earlier sessions, absent in later ones, configuration unchanged throughout.

That order also sorts by what you can act on — 1 and 2 are configuration you control, 3 is a platform constraint, 4 is a budget decision.

> **Sources:**
>
> - Dynatrace SaaS release notes 1.344 (DT docs) — Session Replay unavailability details in Users & Sessions
> - Configure Session Replay Classic for web applications (DT docs) — *"Actual percentage of sessions recorded with SR = cost and traffic control for RUM × cost and traffic control for Session Replay"*; *"If Session Replay is enabled and no URL exclusion rules are defined, all pages are recorded with Session Replay by default."*
> - Technical restrictions for Session Replay Classic for web applications (DT docs)

5. Correlating Replay with Performance

Session Replay is most powerful when correlated with performance data. Use DQL to find sessions that experienced specific performance issues, then review the replay to understand the visual impact.

```
// Long sessions with replay (session duration, not load time)
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(characteristics.has_replay) and characteristics.has_replay
== true
| fieldsAdd duration_sec = duration / 1s
| filter duration_sec &gt; 120
| fieldsKeep dt.rum.session.id, frontend.name, duration_sec,
user_action_count, error.count, geo.country.name
| sort duration_sec desc
| limit 10
```

> **Tip:** Copy the `dt.rum.session.id` value from query results and paste it into the Dynatrace Session Replay search to jump directly to the replay.

6. Identifying UX Issues via Replay

Session Replay helps identify qualitative UX issues that metrics alone cannot reveal:

UX Issue	What to Look For in Replay
Confusing navigation	Users going back and forth, visiting the same pages repeatedly
Broken forms	Users attempting to submit forms multiple times
Invisible errors	JS errors that silently break functionality
Misleading UI elements	Users clicking on non-interactive elements
Layout shifts	Content jumping around during page load
Slow feedback	Long pauses after clicks before visual response

Replay Investigation Workflow

1. **Identify problematic sessions** via DQL (errors, slow sessions, abandonment)
2. **Watch the replay** in Dynatrace UI
3. **Correlate with waterfall** — Performance tab shows timing alongside replay
4. **Note the pattern** — Is this a one-off or a systematic issue?
5. **Quantify with DQL** — How many sessions are affected by this pattern?

```
// Abandoned sessions with replay – users who started but did not finish
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(characteristics.has_replay) and characteristics.has_replay
== true
| filter user_action_count &gt;= 3 and user_action_count &lt;= 5
| filter error.count &gt; 0
| fieldsKeep dt.rum.session.id, frontend.name, user_action_count, error.count,
duration, geo.country.name
| sort error.count desc
| limit 15
```

7. Third-Party Replay Integration

Some organizations use dedicated session replay tools alongside Dynatrace. Common integration patterns:

Integration Approaches

Tool	Integration Method	Use Case
Dynatrace Session Replay	Built-in — no integration needed	Full-stack correlation (backend + frontend)
LogRocket	Session ID correlation via session property	Product analytics focus
FullStory	Session URL in custom session property	UX research focus
Hotjar	Heatmaps + recordings complement RUM data	Heatmap analysis

Correlation Pattern

To link Dynatrace sessions with a third-party replay tool:

1. Capture the third-party session ID via the RUM JavaScript API:

```
// Example: send LogRocket session URL to Dynatrace
LogRocket.getSessionURL(function(sessionURL) {
  dtrum.sendSessionProperties(
    null, null,
    { "logrocket_url": sessionURL }
  );
});
```

2. Configure a custom string session property in Dynatrace for `logrocket_url`
3. Query sessions with the third-party URL for cross-referencing

> **Note:** Dynatrace's native Session Replay has the advantage of being fully integrated with the performance waterfall, backend traces, and Dynatrace Intelligence — third-

> party tools typically lack this correlation.

8. Summary and Next Steps

In this notebook, we covered:

- **How Session Replay works** — DOM mutation recording, not video
- **Data privacy** — Masking levels, HTML attributes, CSS selector rules
- **Configuration** — Sampling rates, conditional capture, storage considerations
- **Finding replays** — DQL queries to locate replay-eligible sessions
- **When a session has no replay** — the five ineligibility causes, and the sampling → enablement → browser → quota elimination order (SaaS 1.344 surfaces the reason directly)
- **Performance correlation** — Linking slow/error sessions to replay
- **UX issue identification** — Patterns to look for in replay analysis
- **Third-party integration** — Connecting Dynatrace with external replay tools

Next Steps

- **WEBRUM-08: Dashboards and Alerting** — Build RUM dashboards with Apdex and set up performance alerts

References

- Session Replay (DT docs)
- Session Replay with strong privacy requirements (DT docs)
- Configure Session Replay for web applications (DT docs)
- Technical restrictions for Session Replay Classic for web applications (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.